

MSc thesis in Geomatics

Spatio-temporal Transformers for Wildfire Prediction

Carlo Cordes

June 2026

A thesis submitted to the Delft University of Technology in partial fulfillment of the requirements for the degree of Master of Science in Geomatics

Carlo Cordes: *Spatio-temporal Transformers for Wildfire Prediction* (2026)

© This work is licensed under a Creative Commons Attribution 4.0 International License. To view a copy of this license, visit <http://creativecommons.org/licenses/by/4.0/>.

The work in this thesis was carried out in the:



Geo-Database Management Centre
Delft University of Technology

Supervisors: Dr. Azarakhsh Rafiee
Justin Schembrie

Co-reader: tbd

Contents

1. Methodology	2
1.1. Workflow & System Overview	2
1.2. Dataset Construction	2
1.3. Model Architecture	4
1.4. Training Routine	10
1.5. Experiments	14
Bibliography	16

TODOs

TODO 1	work in progress	2
TODO 2	Update figure of timestamp configurations	2
TODO 3	Complete model architecture diagram and reference	4
TODO 4	Figure of reflective patch padding	4
TODO 5	Figure of conv with stride equal to patch	4
TODO 6	Update	8
TODO 7	Specify number of heads, number of blocks	8
TODO 8	Diagram of attention heads	8
TODO 9	Fill head & block values	9
TODO 10	Ammend fusion part, explain convolution	9
TODO 11	Update with final params	10
TODO 12	Enhance descriptions	14

1. Methodology

This section provides an overview of the methodological choices made in the investigation. First, a short account of the system overview is given, before an account for the construction of data into datasets is made. Further, a complete walk-through of the model architecture is given. Finally, the integration of the model into a training workflow is explained.

1.1. Workflow & System Overview

work in progress

```
data:
  spatial_extent:
    longmin: -9.4900
    longmax: -6.0234
    latmin: 36.812
    latmax: 42.2724
  crs: "EPSG:3763"
  scale: 1000 # meters
  temporal_extent:
    start_date: "2020-01-01"
    end_date: "2024-12-31"
    day_interval: 1
  sequence_period: 4 # Time between respective sequences
  sample_extent: 10 # Total time of sample
  sample_period: 1 # Time between images in sample
  target_extent: 4
  lead_time: 0 # Time between last sample and target
```

1.2. Dataset Construction

1.2.1. Timeframe Configuration

A section of the research objective is to assess the influences of the temporal scopes of both data and predictions. Both can be variably adjusted depending on the desired test case. As we handle temporally dynamic datasets we need to define how timesteps of samples are represented in the dataset. To ensure consistency in naming these, we here declare a series of term definitions relevant for this step:

- **Sample:** Input data the model sees to form predictions; describes the set of timestamps for one forward pass
- **Target:** The collection of images, that is variable length, that is used for testing the model predictions
- **Sequence:** A pair of a sample and target
- **Dataset:** A collection of sequences
- **Sequence Period:** Separation between individual unique sequences
- **Extent:** Used both sample and target, defines their respective variable length
- **Lead Time:** The time between the last seen timestamp in the sample and first one in the target. In other words: the amount of time the model predicts into the future

Update figure of timestamp configurations

Every sample is constructed by 3 different kinds of data, that are read, packaged together and channeled into the model:

- **Static:** equivalent in every sample, describes non-variable data (e.g. terrain)

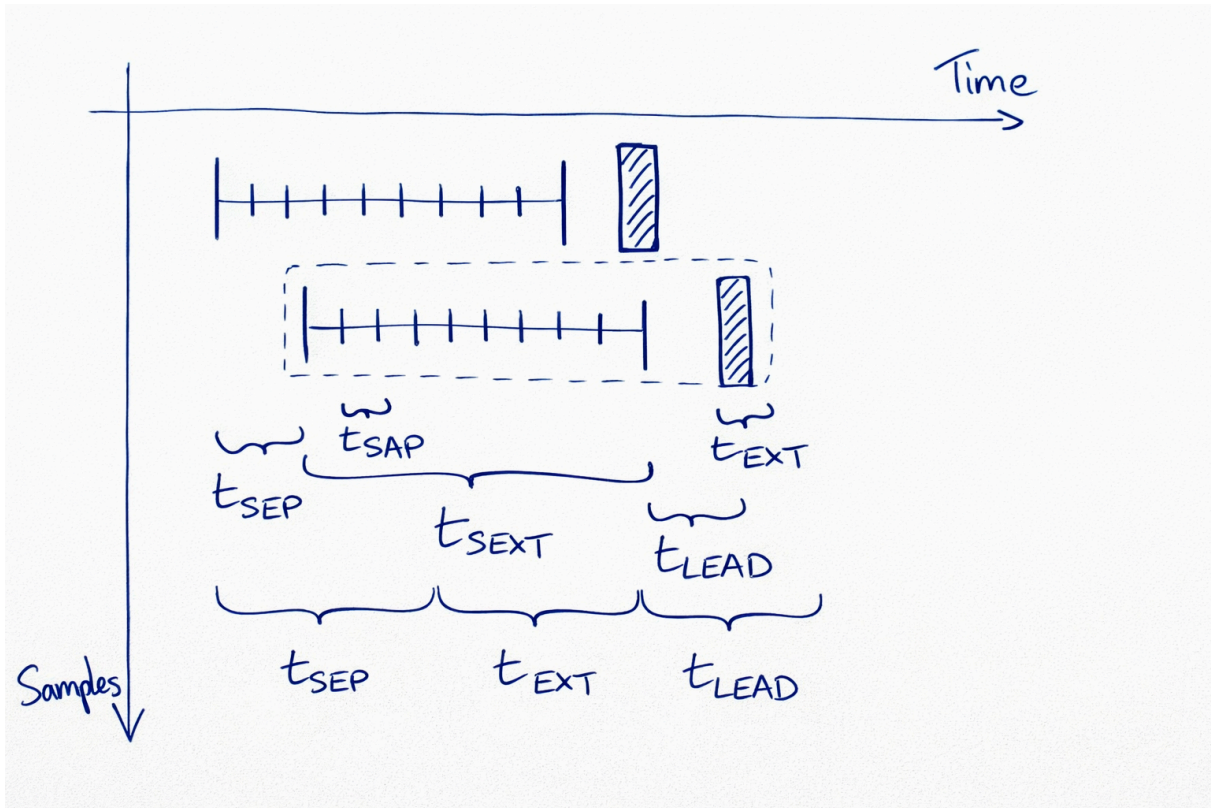


Figure 1.1: Definitions of temporal scope in project datasets

- Dynamic: time-series data consisting of multiple images (e.g. wind)
- Single-dynamic: dynamic data that is considered to be less temporally dependent and assumed static over the extent of the sample (e.g. 30 day precipitation sum)

Figure 1.1 illustrates the terms defined above. It shows two consecutive sequences as well as their respective samples and targets. It shows the temporal division of the sample into subsequent timesteps. The distance between sample and target corresponds to the above definition of *lead-time*.

The configuration file that belongs to a model training governs the total time of interest and all parameters defining the temporal relationship of input data. The dataset builder then generates a set of timestamps as governed by the rest of the temporal configurations and iterates through the respective file for every module. Sequences with missing images are skipped entirely to ensure consistency. These dynamic modules are accompanied by the static data, which is the same in every sample. The result is a set of sample/target combinations spanning the entire domain of temporal interest.

1.2.2. Torch Datasets & Tensors

Moving data from images into the pytorch framework is done using its fundamental data structure: the **Tensor**. Like in the traditional mathematical sense this is an array of arbitrary dimension containing elements of a single data type, but has a few computation-driven features. Most importantly, it is optimized to be handled by GPU. Tensors are directly associated with the model instance (elaborated in Section 1.4.2) and have an attribute that defines whether they are adjustable model parameters. This concept is specifically designed for those tensors containing model weights. PyTorch can track the operations imposed on a tensor, which makes later backpropagation to these easier (Pytorch-Contributors, 2019a). The sequences described

in Section 1.2.1 are appended into such a tensor structures and then saved as portable datasets to be made accessible on larger compute resources.

1.3. Model Architecture

In order to motivate the structure of the here applied transformer, this section walks through the sequential steps incoming data takes through the model. This represents one forward propagation. The later showcased training regiment makes this forward pass once for every batch in the training data, thus cycling through it an abundance of times.

1.3.1. Spatial Embedding

The initial objective of the transformer model revolves around turning input data into tokens via patch embedding. The patch size, a model hyperparameter, defines the pixel extent of square patches every input image is divided into. As we handle only single-band data we treat the image as a grid of height and width: $h \times w$.

This is realized via a 2D convolution with stride equal to the desired patch size. This ensures zero overlap between neighboring patches. An important element to consider is that both height and width of images need not be divisible without rest by the patch size. To counter this we use reflective padding, a process which mirrors input images around their edges to fill in pixel data until the next complete patch. Extending height and width to h_{pad} & w_{pad} allows for perfect image division into patches. As the model input consists of multiple modules M for consecutive time steps t that are processed in batches B of variable sizes the input tensor has dimensionality:

$$\bar{T} \in \mathbb{R}^{B \times M \times t \times h_{pad} \times w_{pad}} \quad (1.1)$$

Every image of dimension $h_{pad} \times w_{pad}$ is then subject to the flattening operation, here illustrated with a patch size of 16 pixels.

$$P \in \mathbb{R}^{16 \times 16} \rightarrow P_{flat} \in \mathbb{R}^{256} \quad (1.2)$$

Up to this point, pixel values have not changed, simply the indexing is altered. To produce the embedding, the convolution operation then maps every flattened patch into the embedding dimension. Here we showcase the operation for a patch size of 16 pixels, mapping to the embedding of dimension 128:

$$e = wP_{flat} + b \quad (1.3)$$

where

$$w = \begin{pmatrix} w_{1,1} & w_{1,2} & \dots & w_{d_{patch},1} \\ w_{2,1} & w_{2,2} & \dots & w_{d_{patch},2} \\ \vdots & \vdots & \ddots & \vdots \\ w_{d_{embed},1} & w_{d_{embed},2} & \dots & w_{d_{patch},d_{embed}} \end{pmatrix} \in \mathbb{R}^{128 \times 256} \quad (1.4)$$

The convolution weights are learnable and are only shared between samples of the same module. Every module is assigned an independent convolution operation and individual weights.

1.3. Model Architecture

1.3.2. Temporal embedding

The model architecture demands a separate routing of static and dynamic data channels. All dynamic modules with a more than one time steps receive a temporal encoding that is an extension to the one described in Section 1.3.1. The objective of this is to give patches that describe the same physical area in different time steps a closer connection to one another. This is done by treating the different time steps for one module together. The single-image 2D convolution is replaced by a 3D convolution that treats the series of images of timesteps t as a 3-dimensional set of values and projects it into the embedding dimension d_{embed} :

$$\mathbb{R}^{t \times d_{patch} \times d_{patch}} \rightarrow \mathbb{R}^{d_{embed}} \quad (1.5)$$

The convolutional operation is analogous to Equation (1.3) for a static embedding with a kernel of dimension:

$$w \in \mathbb{R}^{d_{embed} \times 1 \times P^2} \quad (1.6)$$

In essence, this steps results in a token for every patch in every timestep of every image. This effectively patches every timestep in every module independently. The objective here is to prevent the immediate blending of time steps, as this is the objective of the transformer layers at a later stage.

1.3.3. Spatial Embedding

Having obtained a set of embeddings for all images of varying modalities in the model input, this part of the transformer architecture intends to introduce spatial and temporal relationships between the patches called *embeddings*. These greatly influence the magnitude with which patches learn from each other.

The spatial embedding is applied to every patch regardless of it having a temporal dependency or not. The idea is to assert a learnable idea of proximity from one patch to another. As wildfires are localized phenomena, the the objective of the spatial embedding here is to give neighboring patches a strong correlation. Distant patches, while sharing lower correlation, can still influence one another, with limited effect. The important difference to a convolution here is that we do not simply rule out distant relationships, but keep them contained. Rather than imposing a relationship on the transformer, we give it rough guidelines and freedom to give high weight to the relationship even between distant patches. Having created a notion of spatial similarity is crucial for the later attention process in which more similar patches will conduct higher weighted attention with one another.

In order to create such a relationship we come up with an individual positional encoding for every patch that is added onto each embedding:

$$\vec{e}_{token}' = \vec{e}_{token} + \vec{e}_{pos} \in \mathbb{R}^{d_{embed}} \quad (1.7)$$

To create a positional embedding that retains similarity between proximal patches, we use sinusoidal positional embedding. By imposing sine and cosine waves of varying frequencies over the patch grid, we are able to establish a unique encoding for every patch that has distinct relationships to other patches, depending on their euclidean distance. This is done by first defining a set of frequencies with $d = \frac{d_{embed}}{4}$:

$$\omega_i = \frac{1}{10000^{\frac{i}{d}}} \quad (1.8)$$

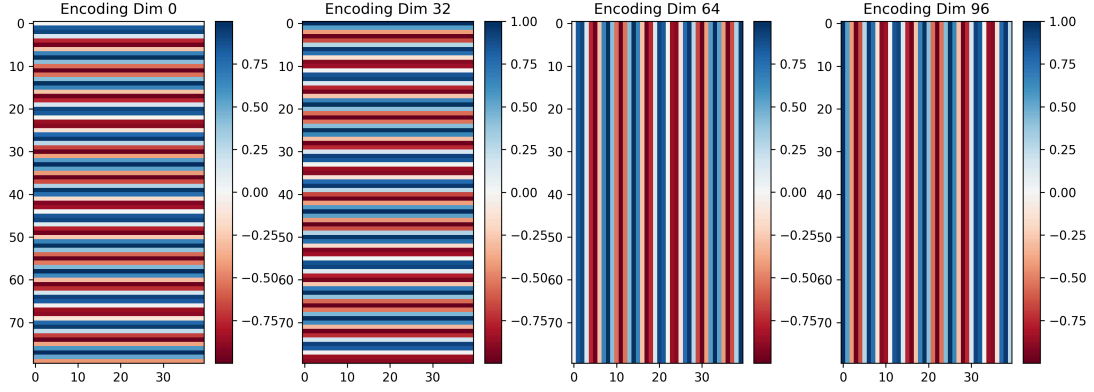


Figure 1.2: Additive encoded values per encoding dimension

The nature of the integer 4 is that in order to create the most unique positional embeddings, we split the added partial embeddings into both height and width and also sine and cosine categories. We scale the frequencies with 10000 in order to fit multiple periods into the image. Interpreting height and width as coordinates we can thus describe the positional encoding of a patch as the vertical concatenation of these frequencies:

$$\vec{e}_{\text{pos}} = \begin{pmatrix} \sin(h\omega) \\ \cos(h\omega) \\ \sin(w\omega) \\ \cos(w\omega) \end{pmatrix} \in \mathbb{R}^{d_{\text{embed}}} \quad (1.9)$$

Figure 1.2 shows samples of the modes of the positional embeddings for every patch in the input image, in this case 80×40 . These include waves that propagate in direction of height and width. For every patch we add the embedding values of all dimensions.

Given the multitude of modes we can form very individualised encodings for all patches. Figure 1.3 shows the cosine similarity of patches to one another for different embedding dimensions. Here the central patch serves as an anchor-patch, with the values of all patches describing the its similarity to it. This shows that sinusoidal positional embedding is able to establish similarities and differences between patches, depending on their respective proximity. Furthermore, it is important to mention that the utility of this process heavily depends on the embedding dimension chosen. A higher embedding dimension allows to pass more sinusoidal modes and hence to create a more distinct characterization of each patch. We here show the cosine similarity for three embedding dimensions, which are more descriptive for larger values.

1.3.4. Temporal Embedding

Weighing like patches of the same modality against each other should be entirely up to the transformer model. Afterall it is the idea to find nuanced temporal relationships both within a modularity and between pairs of them. We therefore do not constrain the temporal relationships with weights but much rather provide an additional encoding of time which is added to every token that stems from a dynamic module. Used here is an encoding of the julian day, which is simply an integer number of the day of the year (0-365). To make use of this in the best way, we encode the julian day so that it captures the true nature of the climatological conditions: its cyclical nature. The embedding of a patch from a day in December should be close to one in the beginning of January, despite their julian day being far apart.

1.3. Model Architecture

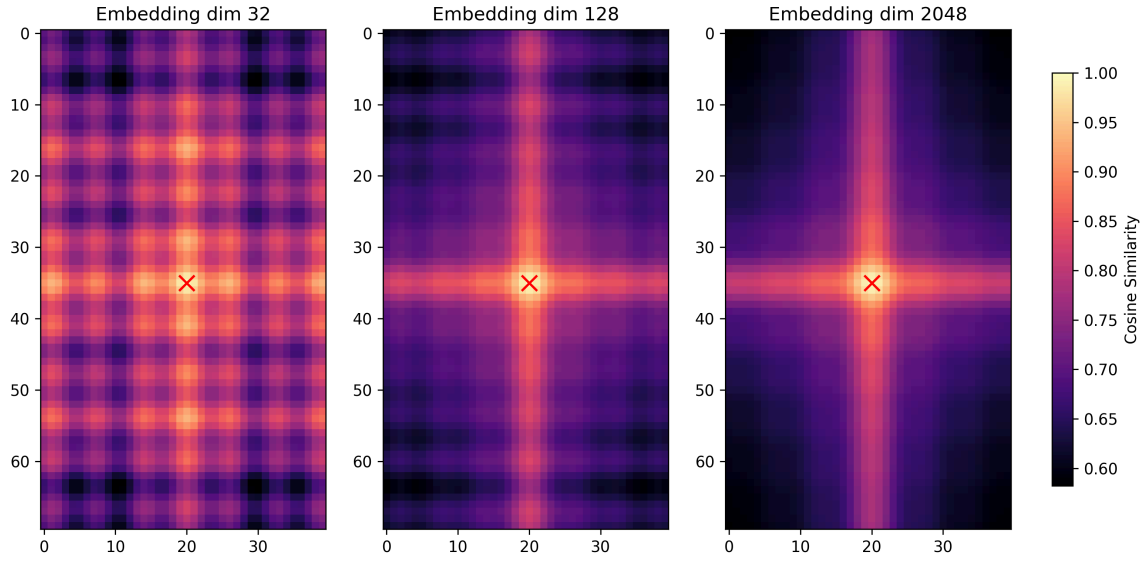


Figure 1.3: Cosine similarity of patches relative to central anchor-patch at varying embedding dimension

Making such an embedding is realized through a two-step process, first mapping the julian day J_t into a clock-like cyclical two-dimensional space and then translating it to the embedding dimension. The translation into a cyclical representation v_t is done using a sinusoidal projection via:

$$v_t = \left[\sin\left(\frac{2\pi J_t}{365.25}\right), \cos\left(\frac{2\pi J_t}{365.25}\right) \right] \quad (1.10)$$

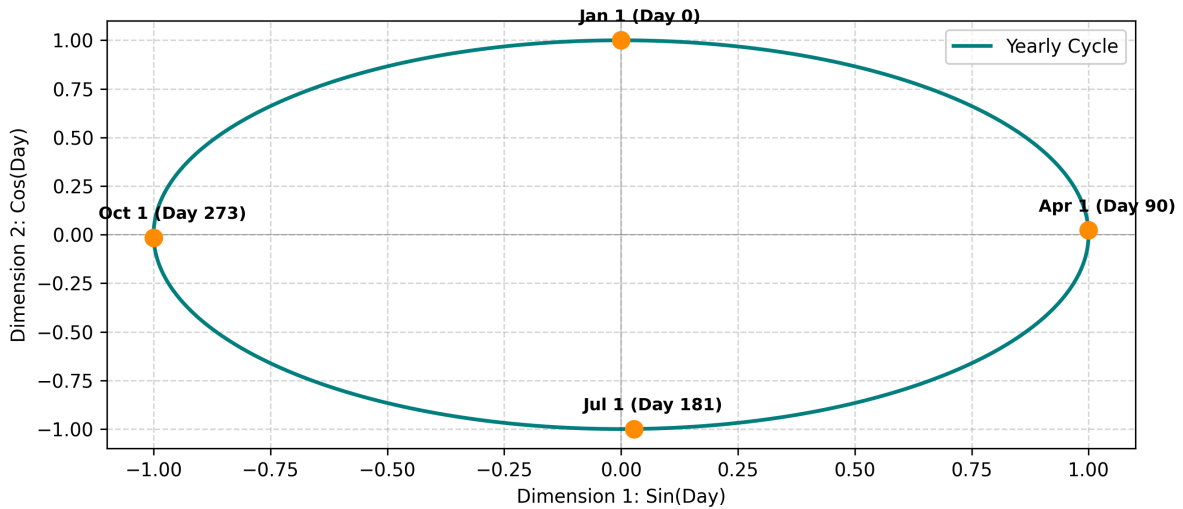


Figure 1.4: Julian Day encoding into 2D vector space

Figure 1.4 shows the representation of each julian day in a two-dimensional vector space, effectively capturing cyclical relationships of months even with distant julian days. The next step entails moving this to the same embedding dimension which the tokens occupy. Similarly to convolution processes shown earlier, we apply a learnable weight matrix to this vector representation projecting these values into $\mathbb{R}^{d_{\text{embed}}}$:

$$\vec{e}_t = W_{2 \times d_{\text{embed}}} \cdot \vec{v}_t + \vec{b} \quad (1.11)$$

Having embedded all tokens of different modalities into embedding space, we additionally give each token a modality tag $\vec{s}_m$. A tag is unique per module and is a learnable parameter with the objective to give the attention heads a clearer idea of where tokens came from. Throughout the learning process the transformer will be able to differentiate between different modules as the tag injects a numeric meaning to all tokens.

The final transformer-ready embeddings for static and dynamic tokens are additions of their individual spatial, temporal and static embeddings like so:

- Static Embedding: $\vec{z}_n^m = \vec{e}_{\text{token}, n}^m + \vec{e}_{\text{pos}} + \vec{s}_m$
- Temporal Embedding: $\vec{z}_{n,t}^m = \vec{e}_{\text{token}, n}^m + \vec{e}_{\text{pos}} + \vec{e}_t + \vec{s}_m$

Simply adding large vectors to our representations of patches might seem counter intuitive at first, however it is crucial to remember that the representations of data modularities are learnable. The longer the transformer architecture trains, it knows to contextualize a patch of precipitation data differently, depending on the time of year. The most elegant analogy to this process is music. If the raw encoding of a patch are analogous to a melody, the positional and temporal encoding represent the harmony accompanying it. Music is propagated as the sums of interfering frequencies, however they are discretely distinguishable to the human ear. The same is true for the transformer, it learns to contextualize the raw encoding (melody) given an extra set of information, the embeddings (harmony).

1.3.5. Attention Layers

Having obtained a set of tokens from the batch of the dataset, the next step dictates how these tokens communicate information between each other and consequently update their embeddings. The core principles demanded from this step are:

- Module isolation
- Shallow self-attention

The idea of keeping the modules in isolation at first is to give later cross-attention process a richer input. By allowing e.g. a patch embedding stemming from the DTM to understand not only itself but its surrounding patches, we save computational cost for later steps as this module delivers an already pre-trained version of every patch. Model weights in this step are only shared between and trained by embeddings of the same module. The idea of keeping this initial self-attention shallow is motivated by the goal of the model. The objective is not for every set of module tokens to best understand its surroundings, but much rather get an initial idea of it, to carry it further into the next model step.

1.3.6. Fusion Layers

With a set of pre-attended tokens we now move onto the fusion step, which represents the first time tokens from different modules are exposed to each other. The traditional implementation with this goal in mind is to use cross-attention heads in which the query vector belongs to the first, and both key and value vector belonging to the other. This step then fuses patch information accross modules.

The methodology chosen here for cross-attending such data is implicit. Rather than taking every token on its own and letting it communicate to all other tokens of all datasets we simplify

the process, as the computational complexity of such a module would scale exponentially with the amount of patches and the amount of datasets as input. The objective here is to rule out potentially unnecessary attention computations e.g. two patches describing vegetation stress and wind that lie far apart in the area of interest. To motivate the following simplification, a short revisit to the core idea of the original objective of cross-attention for translation learning tasks. A sentence, in which every word forms a token, will likely be rearranged in the translation to another language (see Figure 1.5). In the english language we expect the embedding of *red* to have a large influence that of *fire* and likewise in its french counter part. When cross-attending these tokens of the two datasets, we expect no consistency in the arrangements in the positions of directly translatable words. Therefore, a cross-attention mechanism considering all tokens is mandatory.

EN: The red fire spreads accross the land FR: Le feu rouge se propage à travers le pays

Figure 1.5: Example of token order of two cross-attending datasets

This concept is however not consistent in the vision transformer. When dealing with datasets that embody the same area of interest and in the same order, we can confidently expect token 1 to describe the same entity in both images. This very effect allows for an elegant simplification of cross-attention with which we can greatly constrain cross-attending tokens and computational cost.

The fusion layer concatenates tokens for every patch and stacks their values into one new embedding vector. The model thus holds all information later used for prediction in only one set of patch tokens. With this set it then performs self-attention or since multiple modalities are used a quasi-cross-attention block. Since every token in one module cross-attends to every other token in another module very similarly, this step simply carries out this operation together as we expect these token pairs to weight on each other quite similarly. This part of the encoder carries the weight of deep self-attention with n blocks of x attention heads each. The product of these attended tokens then represent the final embeddings of each patch in the vector space that characterizes the prediction of ground truth. Tokens with similar physical properties are represented by similar locations in this embedding space.

Fill head & block values

Ammend fusion part, explain convolution

1.3.7. Decoder Layer

The final task of the vision transformer is to take the embedded vector representations in the embedding space and project them back into a form which represents our ground truth: a 2D image. To clarify, the tensor at this point carries an embedding for each patch, which needs to be projected back from embedding space into 2D:

$$T_{\text{encoder}} \in \mathbb{R}^{B \times n_{\text{token}} \times d_{\text{embed}}} \quad (1.12)$$

It is of course possible to make predictions for every patch. This is however not just an oversimplification but does provide the opportunity to predict for as many samples as possible. Table 1 shows the complete structure of the decoder process that projects back to image space:

Step	Tensor Size $\bar{T}$	Description
Input	$B \times n_{\text{token}} \times d_{\text{embed}}$	Output of attention blocks
Reshape 1	$B \times d_{\text{embed}} \times h_{\text{pad}} \times w_{\text{pad}}$	Reshape back to 4D feature map
Convolution 1 + Batch-Norm / ReLU	$B \times \frac{d_{\text{embed}}}{2} \times h_{\text{pad}} \times w_{\text{pad}}$	3×3 kernel with padding, reduce channel depth
Upsample 1	$B \times \frac{d_{\text{embed}}}{2} \times 4h_{\text{pad}} \times 4w_{\text{pad}}$	$4 \times$ scale, bilinear interpolation
Convolution 2 + Batch-Norm / RELU	$B \times \frac{d_{\text{embed}}}{4} \times 4h_{\text{pad}} \times 4w_{\text{pad}}$	3×3 kernel with padding, reduce channel depth
Upsample 2	$B \times \frac{d_{\text{embed}}}{4} \times 16h_{\text{pad}} \times 16w_{\text{pad}}$	$4 \times$ scale, bilinear interpolation
Convolution 3	$B \times 1 \times 16h_{\text{pad}} \times 16w_{\text{pad}}$	1×1 kernel, pixel-wise classifier
Slice	$B \times 1 \times h \times w$	Un-pad operation to original input size

Table 1: Step-wise decoder mechanism & prediction head

1.3.8. Model Parameters

Module Name	Parameter Count
static_embeds	49,280
dynamic_embeds	57,472
join_backbone	1,586,176
sequence_projector	16,512
decoder	92,481
Standalone Parameters	1,536
TOTAL	1,803,457

Table 2: Model Parameters per class

1.4. Training Routine

The setup of the training regiment of models is crucial as without certain guidelines, the model could make faulty predictions. The most important vocabulary in this regard is *overfitting* and *generalization*. Generalization describes the ability of the model to transfer its learned patterns from training data to previously unseen data. This showcases the ability to concretely learn the physical underlying the data, rather than simply memorizing it. A model is said to be overfitting, if the prediction results rely on the training data including all its inaccuracies and

Update
with final
params

1.4. Training Routine

noise rather than actual representation of data. The training routine used herein caters to both of these issues. As a guide, Algorithm 1 shows the pseudo-code implementation of the training regiment.

To promote generalization of a trained model, we use a train-validation-test split of 70-15-15 respectively. These are the proportions the dataset is split into at the beginning of training. An important note here is that the neither of the subsets of the data are shuffled, which usually is standard for such training processes. This

```
1 Input: Load dataset, initialize dataloader, move to GPU
2 Calculate class imbalance positive weight  $w$ 
3 Initialize loss function, optimizer, model from config
4 Split data (70/15/15) into training- validation- and testing sets
5 best_loss =  $\infty$ 
6 for epoch in num_epochs:
7     train_loss = 0
8     for batch in train_data:
9         Predict for batches given static & dynamic inputs
10        Compute losses based on WBCE, predictions & ground truth
11        Set optimizer gradients to zero
12        Backward pass
13        Update weights
14        train_loss += batch_loss
15    Compute: validation loss on validation dataset
16    if validation_loss < best loss:
17        best_loss = validation_loss
18        Save model
19    if no improvement for 7 epochs:
20        break
21    Compute: test loss on test dataset
```

Algorithm 1: Training routine

1.4.1. Loss Function

Succeeding a successful forward pass, we evaluate the accuracy of the model by comparing its prediction to the ground truth as defined by a loss function. The main specification the loss function has to adhere to in this problem set-up is the type of target domain in its binary form (fire / no fire). The other is the lopsidedness of our target categories. Due to the very rare nature of fire events, most of the target data describes no-fire, or to the model: zeros. We therefore choose a categorical loss function that in addition to being sensitive to class imbalances is scaleable by a custom weight, the weighted Binary Cross-Entropy loss function (WBCE). It depends on the model prediction p_i , the ground truth y_i and positive weight w . The loss function sums N comparisons in total, one for every pixel in the ground truth.

$$L = -\frac{1}{N} \sum_{i=1}^N w y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \quad (1.13)$$

This setup allows for customizable weighing of output cases. As a concrete example, Equation (1.13) allows us to treat categorical errors made by the model differently. The penalty given by the loss function should be much higher for an incorrect prediction of high-likelihood of fire than a missed prediction of a fire as the no-fire category dominates in overall frequency. This also prevents the model from simply predicting zero to all images, in which case it would still be correct in the majority of cases, however wildly missing the point of its training exercise.

Figure 1.6 shows the loss plane for all possible prediction/ground-truth combinations and a class imbalance $w = 5$. This illustrates a zero loss for all correct predictions along with the two extreme parts of the domains in which the model has made an error. By predicting less than the actual ground truth the model is consistently penalized harsher than vice-versa.

Parameter choice of the positive weight w is determined by the entirety of the training dataset before training and is then passed as an input parameter. In essence, all ground truth images are summed to yield a balance of categories.

1.4.2. Back Propagation & Gradient Descent

The goal of the training regimen is to produce the combination of model parameters that produces the least amount of loss, more specifically a minimal validation loss. In order to achieve this, the loss is evaluated against all model parameters after every batch. This optimization problem can be thought of a local or global minimum in the *loss-landscape* of $L(w_1, w_2 \dots w_i)$ to a

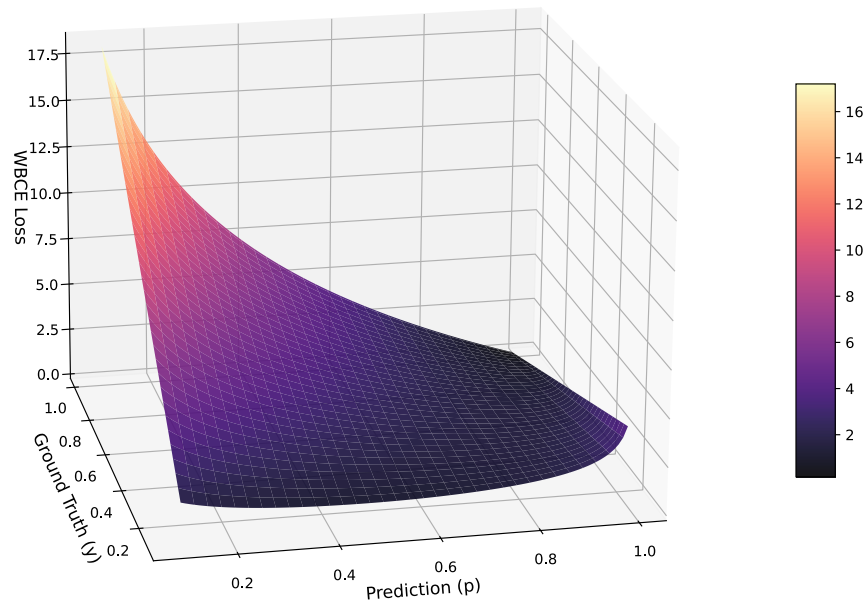


Figure 1.6: Loss surface of Weighted Binary Cross Entropy (WBCE) loss function

1.4. Training Routine

model parameter combination that produces a lower loss, the following steps work to descend along the steepest gradient of this landscape. To do this, we calculate the gradient of the loss due to each model parameter:

$$g_t = \frac{\partial L}{\partial w_i} \quad (1.14)$$

This tries to approximate the extent to which the loss, which describes the discrepancy between prediction and target, is due to this specific parameter. Parameters with a high gradient shall be updated more aggressively than those being responsible for little loss. The method for updating these model parameters is accompanied by a further parameter, the learning rate η . This dimensionless value scales the overall aggressiveness of model updates. It's value heavily influences the stability of the training process. If chosen inadequately, the model may never find an equilibrium state. The updated weight parameters θ are sums of their original values and the loss gradient, scaled by the learning rate:

$$\theta_{t+1} = \theta_t + \eta g_t \quad (1.15)$$

1.4.3. Optimizer & Learning Rate Scheduler

Pytorch uses to further implementation to aid this model weight update process. First is the learning rate scheduler, in this case we use the OneCycle learning rate scheduler. This system is designed to dynamically change the learning rate to a dynamic one that changes with every epoch (η_t). The job of this specific scheduler is to start with a lower learning rate and grant the system so-called warm-up epochs. This caters to one specic problem that is common in machine learning model training and specifically one of this kind which deals with high class imbalance. The model weights, as initialized before the first training loop are random and represent no coherence with the data or reality. Applying a full scale learning rate to these can lead to erratic loss jumps in subsequent epochs. Using the earlier landscape analogy this corresponds to overshooting a downhill section in a landscape. Once the training process has found its momentum, the scheduler will gradually increase the learning rate to a higher value. This is the stage in which the gradient descent process has gained its momentum and we require computed loss gradients to have maximum effects on the model parameters. It is this exact stage in which the model learns most about its tokens. As the loss function approaches its local minimum the scheduler ideally decreases the learn rate again, as not to overshoot the minimum. The effect of this is a user setting and is proportional to the total amount of epochs the system is in training for.

Another crucial implementation aiding training performance is the optimizer. Here chosen was the *AdamW* optimizer. This creates a valuable synergy with the learning rate scheduler to produce and enhanced image of how weights should be updated, following their loss computation. Along with the gradient a weight produces for the loss, the optimizer tracks momentum vectors $\hat{m}_i$ for each weight over the course of a few epochs. The logic insinuates that a parameter improving in the same trajectory for subsequent epochs, will likely continue in the same direction. It can therefore confidently receive a larger update as compared to a parameter that has recently stagnated or fluctuated. Together with the scheduled learning rate the weight updates postulated in Equation (1.15) becomes:

$$\theta_{i+1} = \theta_i - \eta_t \cdot \left(\frac{\hat{m}}{\sqrt{\hat{v}_t + \epsilon}} + \lambda \theta_i \right) \quad (1.16)$$

Another parameter $\hat{v}_i$ tracks the erraticism a parameter shows during epochs. Those values behaving unpredictably will receive suppressed updates while those showing consistency will be rewarded greater updates (Kingma, 2014; Pytorch-Contributors, 2019b).

1.5. Experiments

Enhance
descrip-
tions

1. Varying sample length - how does temporal depth influence climatological contextualization
2. Lead time horizon stress test: how does prediction vary under extensive lead time (future prediction)?
3. Ablation studies - zeroing out weights in embeddings, tracking prediction performance and consequently feature importance

1.5. Experiments

Bibliography

Kingma, D. P. (2014). Adam: A Method for Stochastic Optimization. *Conference Paper at ICLR 2015*. <https://arxiv.org/abs/1412.6980>

Pytorch-Contributors. (2019b). *torch.optim*. pytorch. <https://docs.pytorch.org/docs/2.11/optim.html>

Pytorch-Contributors. (2019a). *torch.tensor*. pytorch. <https://docs.pytorch.wiki/en/tensors.html>

